

III. Advanced AI and Machine Learning Technologies

Classifier Metrics

Because classification is such a widely-applicable concept there are numerous techniques for performing classification. Some classification techniques are extremely general while others are specific to the type of item to be classified. Nevertheless there are common measurements that are used for evaluating classifiers. We'll illustrate these measurements using as an example a (binary) spam filter classifier that assigns each email to one of two classes: "spam" and "not spam". Assume that each email definitely should be assigned to one of the two classes. In other words, there are no emails which might be considered partially spam and partially not spam; each email has exactly one correct class. Therefore, it is at least theoretically possible for a classifier to assign each email to its correct class.

In practice classifiers rarely operate perfectly. One metric for evaluating a classifier is how many items were classified correctly. The percent or fraction of correctly classified items is known as *accuracy*. Accuracy focuses on how often mistakes are made by the classifier. This is an intuitively obvious measure of classifier performance but it has drawbacks. Most notably, it is a poor metric of performance when the items to be classified are not evenly *balanced*, that is, when a collection of items to be classified does not have approximately equal numbers of items in each class. Assume in some collection of emails that the number of non-spam emails is much greater than the number of spam emails: e.g., only 5% of emails are spam. Next, consider a trivial classifier that does not even examine the email in any way but simply classifies every email as "not spam" and never under any circumstances will classify any email as spam. Although this classifier is trivial and clearly broken, it will nevertheless have 95% accuracy because it has classified 95% of all emails correctly. Many phenomena to be classified are highly unbalanced. In many business applications a classifier must classify a phenomena that is undesirable but uncommon, such as customers who fail to renew a subscription, product assembly that is defective, shipments that are likely to be delayed, and financial securities whose price histories suggest they may soon exceed risk limits. For example, for a classifier that identifies customers with suspicious purchases on their credit card accounts, we would expect the vast majority of purchases to not be suspicious. Similarly for a classifier that

identifies Facebook posts mentioning a particular company's products, most posts would refer to other topics. Accuracy would be a poor metric for these kinds of unbalanced phenomena because it may give an unrealistically optimistic evaluation of classifier performance.

Better evaluation metrics than accuracy are derived from separately considering the *false positives* and *false negatives* for each class. For example, we can consider the spam class and ask how many emails that were classified as spam should not have been. That is, how many emails were falsely considered spam: i.e. *false positives* for the spam class. Similarly we can ask how many emails that were not classified as spam but should have been. These emails are *false negatives* for the spam class. It is very useful to consider the rates or percentages of false positives and false negatives, rather than the actual number of emails that were false positives and false negatives for the spam class. The false positive rate is the percentage of all emails that are actually "not spam" which the classifier erroneously classifies as spam. In other words, the false positive rate is the percentage of "not spam" emails which the classifier incorrectly classifies. Similarly, the false negative rate is the percentage of "spam" emails which the classifier incorrectly classifies.

The concepts of false positive and false negative put the focus on the true state of the world, that is, on whether items truly are in one class or the other. One can use a related concept that instead puts the focus on the decisions that the classifier produces. *Precision* is the percentage of items classified as 'positives' that are correctly classified – i.e., are true positives and not false positives. Precision focuses on the output of the classifier - the instances that the classifier has classified 'positive' – and asks about the 'quality' of these positives. For example, of the number of emails classified as "spam", the percent that truly are spam is the classifier's precision.

Recall focuses on the number of items that truly belong to a class, rather than the output of the classifier. *Recall* is the percent of items in a class that the classifier correctly classifies. For example, of all emails that truly are spam, the percent that the classifier considers to be spam is the recall. Precision and recall are much more commonly used in machine learning than false positives and false negatives, though the two pairs of concepts are related.

The ratios defined by precision and recall can be a bit unwieldy. It helps to keep in mind that both are percentages, that is, between 0 and 100% (or equivalently between 0.0 and 1.0), where 100% is the best performance and 0 is the worst. It also helps to keep in mind that *precision* focuses on *false*

positives while *recall* focuses on *false negatives*. False positives reduce the precision score while false negatives reduce the recall score.

In the following example, assume that there are 100 emails, 5 of which are spam emails and 95 are non-spam. This collection of emails is clearly unbalanced since there are 19 times as many non-spam emails as spam emails. Of the 5 spam emails, the classifier thinks that 3 are “spam” but 2 are incorrectly classified as “not spam”. Of the 95 non-spam emails, the classifier thinks that 94 are “not spam” but just 1 email is incorrectly classified as “spam”.

The following table represents this example. Note that for purposes of designating what counts as a false positive, true positive, etc., we must specify the reference class. Here we are using the “spam” class as the reference class, so emails that are (correctly or incorrectly) classified as spam are (true or false) positives. For example, an email that is classified as spam, but is not truly spam, is a false positive.

	Classified as “not spam” (96 emails)	Classified as “spam” (4 emails)
Truly not spam (95 emails)	(true negative) 94	(false positive) 1
Truly spam (5 emails)	(false negative) 2	(true positive) 3

We can calculate each quantity of interest as follows.

$$\text{Accuracy:} \quad (94 + 3) / 100 = 97\%$$

$$\text{Precision:} \quad 3 / (1 + 3) = 75\%$$

$$\text{Recall:} \quad 3 / (2 + 3) = 60\%$$

The formula are more generally stated as follows, using T for True, F for False, P for Positive and N for Negative:

$$\text{Accuracy:} \quad (\text{correct classifications}) / (\text{all emails})$$

$$(TP+TN) / (TP + FP + FN + TN)$$

$$\text{Precision:} \quad \text{fraction of items classified as positives that are true positives}$$

$$TP / (TP + FP)$$

$$\text{Recall:} \quad \text{fraction of items in the reference class that are true positives (correctly classified)}$$

$$TP / (TP + FN)$$

Note in this example that although the accuracy of 97% is very good, the precision is poor and the recall is even worse. Because recall is worse than precision, the false negatives are a worse problem than false positives.

Often for a classifier we want a single score rather than two such as precision and recall. The *F1 score* is a single number that attempts to combine the trade off in precision and recall. F1 score is defined as:

$$\frac{2}{\frac{1}{precision} + \frac{1}{recall}}$$

In the example above, the F1 score is:

$$\frac{2}{\frac{1}{0.75} + \frac{1}{0.6}} = \frac{2}{1.33 + 1.67} = \frac{2}{3} = 0.67$$

The odd computation of the F1 score is specifically defined to emphasize whichever of precision and recall is *worse*. This means the F1 score cannot be manipulated by making one of the two artificially high at the expense of the other: the worse will dominate in the F1 score. This mathematical safeguard against manipulation is necessary because it is *always* possible to design a classifier that produces no false positives or one that produces no false negatives. This of course means that it is always possible to design a classifier that has perfect precision or one that has perfect recall. Take a moment to convince yourself of this by thinking about a trivial and completely non-intelligent classifier that would not at all be useful.

Finally, it is important to note that there can be different costs to a false negative and a false positive because these two types of errors might have very different real world consequences. The costs depends intimately on how exactly one intends to use the decision of the classifier, i.e. what actions are taken based on the class that an item is assigned to. Deciding what cost to assign to a false positive and a false negative is an extremely important business decision. If a false positive is much more costly error than a false negative, we would try to design the classifier to have fewer false

positives, even if that would be offset by more false negatives. For example, imagine an application in which emails that are classified as “spam” are automatically placed in a spam folder and not shown in the user’s inbox. The cost of a false negative, i.e. a truly spam email being classified as not spam, is that the user is annoyed by having spam in their inbox and then having to manually delete it. On the other hand, the cost of a false positive, i.e. a truly legitimate email being classified as spam, is that the email is placed in the spam folder and is not visible in the user’s inbox. If an important and time-sensitive email is not seen by the user, the cost of a false positive may be much greater than the cost of a false negative. This difference in costs of false positives and false negatives might warrant a design that emphasizes as few false positives as possible, even though that would be offset by many more false negatives.

Regression Metrics

In a regression task, the AI model produces numbers representing quantities such as prices, distances, probabilities, or economic data. When the number produced by the model differs from the desired number, the difference between the two is generally called the *residual*. More specifically, there is a convention as to which of the two is subtracted from the other. If the residual is defined as:

$$\text{residual} = \text{model output} - \text{desired output}$$

then negative residuals indicate that the model output is too low, and positive residuals indicate that the model output is too high. For example, if the model outputs 9.7 and the desired output is 9.0, the residual is 0.7, or 0.7 greater than desired. Clearly, the closer the residual is to zero the closer the model’s output is to the desired output.

For a collection of decisions by the model there will be a corresponding collection of residuals, each indicating for a particular input how close the model’s output is to the desired output and whether the model is too high or too low. An evaluation of the regression model often takes some sort of average over all the model’s decisions, and thus over all residuals.

Different regression metrics make different use of the residual. One of the simplest is *mean absolute error (MAE)*. MAE disregards whether the residual indicates that the model’s output is too high or too low. The absolute value of the residual gives the magnitude of the error, whether it is too high or too low. The MAE is simply the average of all absolute values of residuals. For example,

assume the model produces three decisions: 0.8 too high, 0.8 too low, and 0.5 too low. The MAE is the average of these three (positive) numbers, or 0.7.

Another common metric is *Mean Square Error (MSE)*. For each output the residual is squared, and the MSE is the average of these squares. For example, assume the model produces three decisions: 0.8 too high, 0.8 too low, and 0.5 too low. The squares of the residuals are 0.64, 0.64, and 0.25. The MSE is the average of these three squares, or 0.51. Because residuals are squared, larger residuals (i.e. larger errors by the model) result in a disproportionately greater contribution to MSE than smaller residuals. In other words outliers result in more severe penalties. For example, if the model produces three decisions: 3.0 too high, 1.0 too low, and 0.5 too low, the corresponding squares are 9.0, 1.0, and 0.25. Although the worst residual is three times as great as the second worst, its square is nine times larger. MSE is therefore used in applications where larger model errors should be treated much worse (nonlinearly worse) than smaller errors. Stated another way, reducing MSE typically results in reducing the most severe errors at the expense of tolerating many smaller errors. Finally, note that since MSE squares the residuals it disregards whether the output is too high or too low, as MAE does.

Because residuals are squared, the resulting units of the MSE are squared. For example, if the model outputs and desired outputs are dollars, the MSE represents dollars squared, a very odd and unintuitive metric. This makes it difficult to explain the MSE in terms of the expected or typical model output. It would be unhelpful to say “on average each prediction by our model is off by 0.51 square dollars”. For this reason the *root mean square error (RMSE)* is used. RMSE is simply the square root of the MSE, so the units of the RMSE are the same as the units of the desired output, e.g., dollars. Therefore a statement like “RMSE is 0.71 dollars” is equivalent to “on average each prediction by our model is off by 0.71 dollars”.

All three metrics, MAE, MSE, and RMSE, are either zero if the model produces perfect results and greater than zero if the model is not perfect. Also, all three metrics depend on the scale of the data. If we are predicting housing prices which can range from \$50,000 to \$2,000,000, an MAE or RMSE of \$1,000 most likely means a model that often produces numbers very close to the desired output. On the other hand, if we are predicting the shares of a financial instrument that we expect to range from \$1 to \$5 an MAE or RMSE of \$1,000 would almost certainly be useless. Therefore these

metrics cannot be used (without modification) to make meaningful comparisons across different data sets or applications that have different ranges of outputs.

Errors and Loss Functions

In machine learning we train a model using training data, typically over a large number of iterations. The training algorithm repeatedly adjusts the model in an attempt to have the model produce “good” outputs across the training data. With each iteration the learning process tries to reduce errors across the training data. We must specify exactly how to quantify errors so that the model can reduce this quantity at any iteration. A *loss function* (also called a *cost function*) is simply the way that we compute the total error produced by the model for some data set. Therefore the learning process is the process of minimizing the loss function we have selected. Some common loss functions are RMSE or MAE, described above. A loss function can be described as how much to penalize wrong answers by the machine learning model. A loss function can alternatively be described as a measure of what the model considers to be the best set of answers when comparing potential answers that the model could produce. The answer that results in the smallest amount of loss, i.e., the smallest value for the loss function, is by definition the best answer as far as the model is concerned.

Different types of errors can have very different business consequences or costs. Since the training process minimizes the selected error metric (loss function), we should align the loss function with the business impact / cost of different types of errors. Two significant considerations are (a) the treatment of outliers, and (b) whether model outputs that are too high are different from outputs that are too low.

If outliers have a disproportionate business cost then outliers should be disproportionately penalized via an appropriate loss function. For example, assume a machine learning model is used to decide how many shares of a stock to acquire. Acquiring other than the optimal number of shares results in some financial loss. However, if far too many or far too few shares are acquired, then in addition to the ‘normal’ financial loss there is also a stock portfolio that violates certain risk boundaries. Such a violation might result in numerous regulatory and other costs. Therefore errors that are too great in either direction or significantly more costly than smaller errors. A loss function that more severely penalizes outliers such as MSE might be appropriate.

In some situations, a model output that is too high is more costly (or less costly) than an output of the same magnitude that is too low. In other words the cost of errors is not symmetric. For example, assume that a machine learning model is used each month to predict how many units of a product will be needed at a store. That number of products is then shipped to the store at the beginning of the month. If too many products are shipped to the store, many will remain unsold until the next month, which may be suboptimal but relatively innocuous. However, if too few are shipped then the product may sell out, customers will be disappointed, and the store's reputation may suffer. Accordingly, a model output that is too low is worse than an output that is too high. The loss function should penalize residuals that are low more than residuals that are too high. For example, during training 'low' residuals could be doubled but high residuals would not. Note that loss functions such as MAE and MSE do not treat positive and negative residuals differently; only the magnitude of the residual is considered

Sequential Inputs and Recurrent Neural Networks

Some machine learning systems process a sequence of inputs and must deliver an output that is based on the sequence. For example, a system might read the sequence of words in an annual report to determine which sentences relate to the firm's expected growth prospects. As another example, a system might receive as inputs the prices over the past year for a stock in order to output a predicted price of that stock tomorrow. In general there are many use cases where we want the output of the neural network to depend on a sequence, or a history, of inputs, rather than on a stationary set of inputs.

Stated another way, we may want the neural network to incorporate information about inputs seen in the immediate past and possibly the remote past. For example, when we read text we need to remember the words we've read in the previous few sentences or paragraphs in order to understand those sentences and to make sense of sentences we are about to read. A neural network that processes sequences must essentially have some form of memory as it processes the sequence of words in the input text. Different types of neural networks do have a form of memory and these neural networks are indispensable in processing sequential inputs such as text, prices, spoken audio, video clips, and time series data.

The basic *feed forward* neural network we've already seen does not have memory. It transforms inputs to outputs, but is not adapted to process sequences of inputs over time. A feed forward network receives one or more inputs at a time and generates one or more outputs. A trained feed forward neural network produces the same output when given the same inputs because the output depends solely on the inputs received at that time. In other words, the neural network has no 'memory' of what inputs it has previously seen, so previous inputs have no affect whatsoever on what output is produced by the neural network; only the current input determines the network's output.

In contrast, many tasks require memory of the inputs previously seen. For example, when determining whether the tone of a news article is positive or neutral, all words in the article are potentially relevant. We read each word from the first in the article to the last, remembering the previous words as we read, eventually coming to a conclusion about the tone of the complete article. Our conclusion about the article's tone evolves as we continue to read more words (i.e., as we receive additional inputs). We draw firmer conclusions about the tone at the end or near the end of the article. In these kinds of sequence-processing tasks the neural network benefits from some sort of memory to store the previously seen inputs.¹ Moreover, the number of inputs the network must remember can be arbitrarily large since there is usually no predetermined limit on the number of inputs in a sequence. The neural network must be flexible enough to handle any length sequence. To address these requirements, the *recurrent neural network* was developed.

In contrast to a feed forward neural network, a recurrent neural network has memory. It does not literally store a copy of every word it has seen – that would not be very efficient. Instead the memory of a recurrent neural network is a set of numbers (i.e., a *vector*) that is adjusted with every new input in the sequence. The numbers in this memory vector must store information that accurately represents the totality of the inputs seen so far, somewhat analogous to maintaining a running average of the inputs seen so far. A network can in principle be designed to have as much

1 It might seem that we wouldn't require memory if neural network instead has a large number of inputs. For example, we could have a separate input to handle every word in a sentence at once, and the neural network would process those words all at once to perform its task. That approach would work if the number of inputs were so large that it exceeded the number of words in the vast majority of sentences. Unfortunately, too many inputs leads to very large networks that are difficult to train. Also, sentences can have very different lengths, so any number of inputs we select for the network would either be too many or too few for most sentences. Finally, when we need to process extremely long sequences, this approach of one network input per sequence input becomes unusable. A neural network with some form of 'memory' offers a much better approach.

memory as desired. This means it can store as many numbers as specified in the design of the network, and having more numbers generally allows more information to be stored. Specifically, more numbers allows the network to store more information about the words previously seen by the network. The network's vector, or memory, can also be called a *representation* of the words that the network has already seen.

A recurrent neural network can output a prediction for every input it receives. For example, as stock prices are received in real time the network can build an internal assessment of what the stock price is likely to be at the close of trading. This prediction will improve as more prices are received so we might choose to ignore or discount the network's prediction until the end or near the end of the trading day. Alternatively, the network can be designed to produce an output only after an entire sequence has been received. For example, the network could process all words in the article one at a time but only predict the tone of the article once at the end.

The most popular type of recurrent neural network is the *LSTM* (“*long short term memory*”). An LSTM processes sequences very well because it is designed to learn very complex patterns in the sequences used to train the network. For example, assume an LSTM has been trained with thousands of sentences of the following form:

“I speak fluent French.”
“I do speak fluent German.”
“I don’t speak fluent Italian.”
“I can speak fluent Japanese.”

Although these sentences are not identical they have obvious similarities. These similarities are a pattern that an LSTM or other recurrent neural network can learn. One advantage of learning these patterns is that the LSTM learns that certain words, such as *French* and *German*, or *fluent* and *speak*, are somehow either related or are similar in meaning. Another advantage of an LSTM learning patterns in a sequence is that the neural network can then learn how to connect different pieces of information scattered across different parts of the sequence. For example, consider the following pairs of sentences:

“I grew up in France. I can speak fluent French.”
“I grew up in Germany. I speak fluent German.”
“I grew up in Italy. I certainly speak fluent Italian.”
“I grew up in Japan. I speak fluent Japanese.”

Although these pairs of sentences are not identical, there is a strong similarity among them in structure and meaning. A sufficiently-trained LSTM could learn this pattern and then use its knowledge of the pattern to infer the following:

- words like “France” and “Germany” are somehow similar (i.e., country names).
- words like “German” and “Japanese ” are somehow similar (i.e., language names).
- the relation between the words “France” and “French” is somehow like the relation between the words “Japan” and “Japanese”.
- when seeing the first few words “I grew up in _____”, the next word is probably the word “France”, “Germany”, “Italy”, or “Japan”, or a word that is similar to one of those words (i.e. a country)

LSTMs are also very good at understanding how parts of a sequence can depend on much earlier portions of the sequence. For example, if a well-trained LSTM sees the sentence

I grew up in France.

followed by five other sentences, and then the first few words

I speak fluent _____

the LSTM could predict that the next word is “French” if it was able to retain this critical piece of information after it reads the first sentence. This ability to accurately predict words demonstrates the ability of the LSTM to “understand” sentences, and which words are *not* likely to belong in the blank. More generally the memory of a neural network allows it to understand the underlying structure of the sequence in some way.

Recognizing complex patterns in sequences is essential in many machine learning applications such as price and sales prediction, text processing, and time series analysis. If a neural network can recognize that a sequence follows a particular pattern, the neural network can then take actions or make decisions based on which pattern the sequence follows. For example, if a neural network is trained to recognize that most sentences with a positive tone follow one of seventeen different patterns, that knowledge can be used to classify the tone of sentences the network has not seen previously.

Self-supervised Learning

Self-supervised learning was previously described as a way to generate labels from unlabeled data so that the newly-labeled data could be used in training a machine learning system. More details on self-supervised learning should clarify this advanced technique and its uses. There are two important aspects of self-supervised learning: how labeled data is created from unlabeled data, and what the machine learning system learns once it is trained with this labeled data.

Creating labeled data

A common way of automatically creating labels without human input from unlabeled data is to remove part of the data and use the removed part as the label. For example, after collecting thousands of paragraphs of text from the Internet it would be simple to randomly delete a few words from each paragraph. Each deleted word is a label, i.e., the correct word for the corresponding blank. We can likewise collect a large number of images and randomly “white out” or delete small blocks from each image. The portion of an image that was whited out is the label for the altered image.

A different manner of generating labels automatically involves sequential data such as the words in a document or the frames in a video. We can consider each element of the sequence to be the label for the elements that precede it. For example, if we have the sentence “I speak fluent French” then the word “fluent” would be the label for the sequence “I speak” and the word “French” would be the label for the sequence “I speak fluent”. Each frame in a video can be the label for the sequence of previous frames.

Yet another way to automatically create labels is to corrupt or transform an input in some way. For example, one can automatically add noise or blurring to an image, substitute a few words in a sentence with randomly-selected words, or randomly alter a few of the frames in a video.

Training objectives

Once the labeled data has been automatically created, it is used as training data for a machine learning system. The specific learning task depends on the data set. When the labels are created by deleting portions of the unlabeled data, a common task is to predict the deleted part. For example, if

a word is deleted from a sentence, the input is the entire sentence except the deleted word. The machine learning system must “fill in the blank” by predicting the missing word. For images the system might need to output (i.e., draw) the portion of the image that was deleted.

Another, somewhat easier task is for the machine learning system to predict which of two or three choices is the deleted part. For example, the input could be the partial sentence “I speak fluent _____”, and the choices could be the words “French”, “car”, and “pencil”. Selecting the correct word from among a small number of choices is much easier than having to simply fill in the blank by choosing any of thousands of potential words. In a video application the system could be asked which of several possible frames has been deleted from the video.

When the labels have been created by changing portion of the input, rather than by deleting portions of inputs, the machine learning system might be trained to identify which portions were changed. For example, if the original sentence “I can speak fluent French” had the second word randomly changed to “I can pencil fluent French”, the system could be trained to predict that the second word was altered. This is tantamount to teaching the system which word does not belong in the sentence.

Why this training is useful

A machine learning system trained to solve any of the above tasks does not seem to have any immediately obvious use. In summary, the training makes the system indirectly useful in performing one or more different but related tasks.

First, consider what the machine learning system must do well in order to perform any of the above tasks. To correctly fill in the missing word in a sentence, the system must at some level understand the text surrounding the blank. For example, to identify the word that belongs in the sentence “I speak fluent _____”, the machine learning system must select a word that would be grammatically correct in that location. More importantly, the system must understand that the word is most likely the name of a language. To properly identify the missing frame in a video, the system must understand something about what the frames immediately before the missing frame show.

In other words, all of these tasks train the machine learning system to learn something about the underlying structure of the inputs, whether it is the structure of English language text, the structure of what people shown in the last seven seconds of video could possibly do in the next frame, or

what a cat looks like so that the missing parts of a cat image can be filled in by the machine learning system.

Next, assume that this training allows the machine learning system to gain enough of an understanding of what the data can look like, such as what sequences of words make sense or what an image of an animal can look like. This understanding is potentially very useful in other tasks. Specifically, the trained machine learning system can be used as a component that makes many other, more interesting tasks much easier to learn. For example, once the machine learning system is trained to understand the structure of English text, it can be used as a component of a system that learns to classify sentences (e.g., does this sentence in an annual report relate to the firm's expected growth prospects) or to generate new text for a story (e.g., which sentences would logically follow the sentence "You wouldn't believe what I saw in the park.") A machine learning system that understands the structure of video can be used in a larger machine learning system that detects objects (e.g., are there any pedestrians crossing the street) or tracks objects (track the path of an aircraft). In other words, the learning on the original task is transferred to a more useful task. In fact this learning can be transferred to many additional tasks simultaneously. The training of the original machine learning model can be amortized over many useful applications.

Generative Adversarial Networks (GANs)

One neural network technique developed fairly recently is known as the *Generative Adversarial Network (GANs)*. GANs are used in the very complex task of generating realistic artificial data such as images, audio, or text. In a GAN two neural networks learn to perform different but related tasks, and in doing so they compete with each other in a [zero-sum game](#). Their competition causes both networks to improve at their respective tasks, with the end result being an output that we will show is quite useful. In other words, the GAN process trains one of the two neural networks to *generate new data* with the same characteristics as the training data. Even better is that the training data is not labeled so it is not costly to acquire.

The GAN process is best described using an example that involves images. The overall goal is for the GAN in this example is to generate new, realistic images of artwork automatically. The (unlabeled) training data is a collection of thousands of pieces of artwork by hundreds of different

artists. There are two neural networks we will call the “forger” and the “detective”. The forger’s task is to create fake artwork. It learns to do so by mimicking the real artwork in the training data. The detective’s task is to learn to distinguish fake artwork from real artwork. There is a third system (a “supervisor”) that feeds data to both networks, receives outputs from both networks, and generally coordinates the training of the “forger” and the “detective”.

The detective is a classifier and its training data is a set of input-output pairs. Each input is artwork, either real artwork or fake artwork generated by the forger. Each time the supervisor randomly selects whether to give the detective a real or fake artwork. The detective’s output is the classification “fake” or “not fake”. Note that we do not need a human to create labels for the detective’s training data since the supervisor knows whether it has selected artwork from the forger or from the collection of real artwork. The training process teaches the detective to identify when artwork is generated by the forger. It will easily learn to do so if the forger generates poor quality artwork that does not resemble the style of the real artwork.

The detective’s process also generates labels about whether the forger’s artwork is good (fooled the detective) or not good (didn’t fool the detective). This means we can give apply the label to every image the forger generates. Again, we do not need a human to apply these labels since the supervisor knows which artwork fooled the detective. The forger is trained using these labels to learn to create more realistic images with a greater chance of fooling the detective. In summary, the detective gradually becomes better at detecting which artwork was generated by the forger rather than a human. This causes the forger to gradually learn how to create better (more realistic) artwork. The end result is a neural network (the forger) that generates new images that look like they were created by a human artist.

Note that the GAN has automatically created a labeled data set for the detective neural network using only the unlabeled data set (real artwork) it was provided. Specifically, the GAN process generates labels “fake” or “not fake” for each image provided to the detective. Similarly, the GAN process generates labels “fooled” or “didn’t fool” for each image generated by the forger. Though our original data is unlabeled, the process creates labels which are used to train both neural networks.

The GAN process can be used to generate realistic images, videos, sound, or text. More formally, given a set of images, text, or any other data, the GAN technique learns to generate new data with

the same structure as the original data. Moreover, the GAN technique allows a semblance of control over the output the GAN generates. For example, a GAN trained to automatically create original artwork in the style of Picasso, Monet, or Caravaggio could be told to generate a dozen images in the style of Monet, or images in a combined style of Monet and Picasso.

Self Assessment Questions

You should spend enough time perusing this section to be able to answer the following questions.

What is the accuracy evaluation metric, and what are some of its shortcomings?

What are precision and recall?

Why exactly is it not helpful to hear that a classifier under consideration produces very few or no false negatives? Likewise, why exactly is it not helpful to hear that a classifier under consideration produces very few or no false positives?

What are some regression metrics? What are their general strengths and weaknesses?

What is a loss function? How does the loss function affect neural network training?

What does a recurrent neural network do? How does it differ from a feedneural network?

What is the goal of self-supervised learning? What can it be used for?

What is the basic use of a GAN?